Brickts SQL course — Week 6 — Explain HVAC analytics

sparql-playground / lessons/sql

Generated for offline / Kindle reading

Week 6 — Explain HVAC analytics

This pack concatenates the six daily lessons and the weekly challenge, then the Week primary-reading links from SOURCES.md. Work the SQL in `psql` against `brickts_course`; keep notes under ignored `student-work/sql/`.

Contents

- Day 36: Day 36 — Choose the denominator for comfort percentage
- Day 37: Day 37 — Use mixing mathematics with physical eligibility checks

- Day 38: Day 38 — Keep BAS and web weather as separate sources
- Day 39: Day 39 — Weight aggregates and define a scatter dataset
- Day 40: Day 40 — Read the shape of fan and temperature relationships
- Day 41: Day 41 — Save query results without hiding freshness or detail
- Day 42 (weekly challenge): Day 42 — Week 6 independent challenge — an analytics engineering review

---

Day 36 — Choose the denominator for comfort percentage

Previous: Day 35 · **Week 6 · 45–75 minutes.** Prerequisites: Days 1–35, or equivalent SQL skills.

Goal

Report temperature comfort as a clearly defined percentage with coverage and operating context.

Before you start

Inspect `zone_temp`, `zone_temp_sp` and `fan_status` in `course.ahu_5m`. The fixture's teaching comfort band is 20–24 °C inclusive. This is an exercise policy, not a universal comfort or health standard.

Concept

A percentage needs a numerator, denominator and eligibility policy. For all-time observed comfort, the numerator is eligible zone-temperature seconds inside the band and the denominator is all eligible zone-temperature seconds. Report unknown temperature seconds separately. For fan-on comfort, both temperature and status must be eligible and status must be on; the denominator changes accordingly.

A setpoint error metric answers another question: `zone_temp - zone_temp_sp` measures deviation from a target. A temperature can be close to its setpoint while outside a chosen comfort band. Keep band compliance, setpoint deviation and measurement availability as distinct columns.

Tiny example

```
SELECT 100.0 * sum(seconds) FILTER (WHERE
         temp_c BETWEEN 20 AND 24)
       / NULLIF(sum(seconds), 0) AS
         observed_comfort_pct
FROM (VALUES (22.0, 300), (28.0, 60)) AS
         v(temp_c, seconds);
```

The result is about 83.33%, because five comfortable minutes and one uncomfortable minute have unequal weights. Averaging two row flags would report 50%.

SQL challenge

Report all-time and fan-on comfort per equipment over the full fixture. Include comfortable seconds, eligible denominator seconds, unknown seconds, the named band and percentage. Define the output for an empty eligible denominator; null with an explanation is preferable to a fabricated 0% or 100%.

Experiment

Use scratch rows at 19.9, 20, 24 and 24.1 °C. Predict inclusion before running. Add a missing temperature during fan-on time and

show why removing it from both numerator and denominator also requires reporting coverage.

Acceptance checks

- North Supply A has one uncomfortable hour: all-time comfort is approximately 97.9167%, and fan-on comfort approximately 95.8333%.
- The other equipment have 100% for both defined denominators.
- Inclusive boundaries pass; a zero denominator does not cause division by zero.
- Output makes fan-on and all-time percentages distinguishable without reading SQL.

Stretch and reflection

What additional measurements would a formal comfort assessment need? Explain why this temperature-only teaching result must retain its limited name.

Save SQL, actual output and a short interpretation in your own `student-work/sql/day36/` directory. Label evidence

observed, **fixture-only**, or **not run**. Consult this week's primary references for exact language and domain details.

Previous: Day 35 ·

Day 37 — Use mixing mathematics with physical eligibility checks

Previous: Day 36 · **Week 6 · 45–75 minutes.** Prerequisites: Days 1–36, or equivalent SQL skills.

Goal

Calculate a teaching outdoor-air fraction and understand when its denominator makes the result unreliable.

Before you start

Read units for `oat`, `rat`, `mat` and `damper_pct`. Temperatures are °C; damper position is a percentage signal. Query metadata rather than trusting an English column label.

Concept

For an ideal two-stream sensible-temperature mixture, `mat = fraction × oat + (1 - fraction) × rat`. Rearranging gives `fraction = (mat - rat) / (oat - rat)`. This simplified

relationship assumes representative, compatible measurements and adequate mixing. When outdoor and return temperatures are nearly equal, a tiny sensor error creates a large fraction error. Declare a minimum absolute temperature separation, such as 2 °C, as a teaching eligibility threshold.

A fraction outside 0–1 should remain visible as a diagnostic result. Clamping to 0–1 can hide sensor or model problems. Damper percentage is actuator position; it is not necessarily outdoor airflow fraction. Fan status and operating mode determine whether interpreting the mixture is useful.

Tiny example

```
SELECT (18.0 - 24.0) / NULLIF(12.0 - 24.0,
         0) AS outdoor_fraction;
```

The result is 0.5, or 50%. This example demonstrates the algebra, not a complete economizer diagnostic.

SQL challenge

Using `course.ahu_5m`, report eligible fan-on intervals with return, outdoor and mixed-air temperature, separation, computed fraction

and damper percentage. Give excluded rows a reason such as missing input, fan off or inadequate separation. Summarize eligible coverage and out-of-range fractions by equipment without labelling them automatically as faults.

Experiment

Hold return temperature at 24 °C and mixed air at 23.9 °C. Compare outdoor temperatures of 12, 23.8 and 24 °C in scratch data. Predict the sensitivity and undefined case before running. Then perturb mixed air by 0.2 °C.

Acceptance checks

- The tiny example produces fraction 0.5, with percentage conversion applied exactly once.
- Equal temperatures never cause division by zero.
- The declared separation threshold appears in report metadata.
- No report substitutes `damper_pct / 100` for the computed fraction or claims the simplified calculation proves an economizer fault.

Stretch and reflection

What happens if one temperature is Fahrenheit while the others are Celsius? Explain why unit validation belongs before the formula rather than after aggregation.

Save SQL, actual output and a short interpretation in your own `student-work/sql/day37/` directory. Label evidence **observed**, **fixture-only**, or **not run**. Consult this week's primary references for exact language and domain details.

Previous: Day 36 ·

Day 38 — Keep BAS and web weather as separate sources

Previous: Day 37 · **Week 6 · 45–75 minutes.** Prerequisites: Days 1–37, or equivalent SQL skills.

Goal

Compare weather measurements and temperature bins without erasing provenance or duplicating HVAC observations.

Before you start

The course fixture contains BAS `oat` points only. It contains no downloaded weather history. Use a small, locally written scratch weather relation for this exercise; mark it synthetic. No external account or network call is required.

Concept

An OAT value needs source, timestamp, unit and location context. A building sensor and a weather website can differ because of placement, averaging, publication delay or measurement method. Neither becomes interchangeable simply because both columns say temperature.

A join must declare its matching policy. An exact timestamp join may lose observations; an unbounded nearest-time join may borrow stale weather. More than one weather row within tolerance can multiply a BAS row and inflate every later denominator. Retain original source timestamps and match age. If multiple AHUs have BAS OAT sensors, choose a site weather policy explicitly rather than averaging them accidentally.

Tiny example

```
SELECT source, floor(oat_c / 5.0) * 5 AS
         bin_lower_c
FROM (VALUES ('BAS', 18.0),
         ('synthetic_web', 20.0),
             ('BAS', -1.0)) AS v(source,
         oat_c);
```

The bins are 15, 20 and -5 °C. Here a bin is `[lower, lower + 5)`; negative temperatures expose truncation mistakes.

SQL challenge

Create a scratch comparison dataset with at least three synthetic weather timestamps, a source label and explicit °C units. Match it to a bounded BAS window under a declared tolerance and tie policy. Report unmatched observations and weather age. Produce OAT-bin summaries of BAS fan-on kW with sample count and valid seconds, retaining source identity.

Experiment

Add a second weather row inside one match tolerance. Predict the row count of a naive join and then show how your chosen policy preserves at most one weather match per BAS observation. Include a timestamp outside tolerance.

Acceptance checks

- Scratch weather is visibly labelled synthetic and remains outside `course` fixtures.
- The negative-temperature example bins to -5 °C.
- Matching does not multiply BAS intervals or reuse weather without a bounded age.

- Every bin names source, boundaries, equipment scope and coverage.

Stretch and reflection

Would aggregating hourly BAS data before joining hourly web data remove all alignment issues? Explain the remaining timezone and averaging-window questions.

Save SQL, actual output and a short interpretation in your own `student-work/sql/day38/` directory. Label evidence **observed**, **fixture-only**, or **not run**. Consult this week's primary references for exact language and domain details.

Previous: Day 37 ·

Day 39 — Weight aggregates and define a scatter dataset

Previous: Day 38 · **Week 6 · 45–75 minutes.** Prerequisites: Days 1–38, or equivalent SQL skills.

Goal

Produce numerically defensible summaries and a precise row contract for a future scatter plot.

Before you start

Reuse your interval and eligibility policies. A plot is optional; the required artifact is an auditable SQL result that could support one. Today uses plain PostgreSQL, with no plotting service or Timescale feature.

Concept

For a temperature or power mean over time, use `sum(value × valid_seconds) / sum(valid_seconds)`. A mean of means is valid

only when the underlying weights are appropriate. Two AHUs with different runtime or coverage should not receive equal time weight accidentally. Conversely, a fleet metric intended to give each equipment equal influence needs that distinct policy named.

A scatter result needs a row grain before it needs colors. Define one row per site/equipment/time interval, an OAT x-axis in °C, power y-axis in kW, matched timestamps, interval weight and eligibility fields. Joining independent point histories on timestamp alone can cross equipment boundaries. A point cloud can reveal associations; it does not by itself establish causation or diagnose a fault.

Tiny example

```
SELECT sum(kw * seconds) / sum(seconds) AS
         time_weighted_kw
FROM (VALUES (12.0, 300), (6.0, 60)) AS
         v(kw, seconds);
```

The weighted result is 11 kW; the unweighted mean is 9 kW.

SQL challenge

Create a scratch view for fan-on OAT-versus-power scatter rows over a bounded window. Include site, equipment UUID, timestamp, OAT, kW, valid seconds and a source label. Produce both per-equipment time-weighted means and a fleet time-weighted mean from the underlying weighted sums.

Experiment

Compare a fleet result assembled from sums and weights with an average of equipment averages. Make an unequal-weight scratch example if the fixture happens to obscure the difference. Predict which equipment dominates the time-weighted result and why.

Acceptance checks

- The tiny example returns 11 kW.
- Scatter rows have a unique declared key and no cross-site label collisions.
- Missing power never appears as a known 0 kW point.
- Weighted sums and weights reproduce the final mean; plotted row count alone is not presented as valid duration.

Stretch and reflection

What would equal-equipment weighting answer that time weighting would not? Name both metrics as a dashboard reader should see them.

Save SQL, actual output and a short interpretation in your own `student-work/sql/day39/` directory. Label evidence **observed**, **fixture-only**, or **not run**. Consult this week's primary references for exact language and domain details.

Previous: Day 38 ·

Day 40 — Read the shape of fan and temperature relationships

Previous: Day 39 · **Week 6 · 45–75 minutes.** Prerequisites: Days 1–39, or equivalent SQL skills.

Goal

Evaluate HVAC relationships without confusing an algebraic illustration with a calibrated equipment model.

Before you start

Inspect the units and ranges of `fan_speed_pct`, `kw`, `sat`, `sat_sp` and `cooling_pct`. Keep fan-off rows, startup behavior and missing values distinguishable in your scratch output.

Concept

A fan affinity-law illustration says power scales approximately with the cube of speed under suitable comparable conditions. Real systems also have static pressure control,

changing resistance, minimum power and equipment efficiency. A percentage command is not guaranteed to be measured speed. Use the cube calculation as an explicitly assumed reference, not a production prediction fitted from these two days.

Supply-air temperature error is `sat - sat_sp` in °C; it is not a percentage. A larger cooling command can accompany a larger temperature error because the controller is responding to a load. That association alone does not show that cooling caused the error. Preserve operating context when preparing comparisons.

Tiny example

```
SELECT speed_pct, 12.0 * power(speed_pct /
         100.0, 3) AS illustrative_kw
FROM (VALUES (50.0), (100.0)) AS
         v(speed_pct);
```

Under that stated 12 kW full-speed illustration, 50% speed yields 1.5 kW. Those assumptions are not fixture calibration results.

SQL challenge

Create fan-on scatter rows for speed-versus-kW and cooling-command-versus-SAT-error. Include equipment, timestamp, units and valid seconds. Compare per-equipment summaries with an all-equipment summary. Explain whether the fixture contains enough speed variation to estimate a meaningful curve; a constant input cannot identify one.

Experiment

Build scratch observations with two equipment operating at different fixed power levels and different temperatures. Compare a pooled trend with within-equipment variation. Explain how equipment identity can create a misleading apparent relationship even when each individual series is constant.

Acceptance checks

- The tiny calculation returns 1.5 and 12 kW, with its assumptions stated.
- Temperature error retains a °C unit; commands retain percent units.
- North Supply B's warm SAT interval is visible on Jan 5 12:00–13:00Z.
- Your interpretation distinguishes a relationship, an illustrative model and

evidence of a fault.

Stretch and reflection

What additional data and controlled comparisons would make a fan power model defensible? Identify one useful validation window that differs from a fitting window.

Save SQL, actual output and a short interpretation in your own `student-work/sql/day40/` directory. Label evidence **observed**, **fixture-only**, or **not run**. Consult this week's primary references for exact language and domain details.

Previous: Day 39 ·

Day 41 — Save query results without hiding freshness or detail

Previous: Day 40 · **Week 6 · 45–75 minutes.** Prerequisites: Days 1–40, or equivalent SQL skills.

Goal

Distinguish a view, a materialized summary and a plot-ready result, with explicit refresh and lineage.

Before you start

Use only the `student` schema for objects you create. Pick one completed daily report and its bounded source window. Keep the SQL text alongside any stored results.

Concept

A PostgreSQL view reruns its query when read. A materialized view stores a result and changes only when refreshed. A table created from a query is also a snapshot, with no

automatic connection to later source changes. These behaviors matter when a dashboard looks current but its numbers describe an older extract.

A summary row per equipment/day and a scatter row per equipment/interval serve different purposes. Storing just means loses individual points, weights and exclusions needed to reconstruct a plot or examine a surprising hour. A summary should carry numerator, denominator, quality counts, window, policy version and a refresh/as-of record appropriate to its use. No metadata column by itself guarantees freshness; the refresh procedure must maintain it.

Tiny example

```
CREATE MATERIALIZED VIEW
         student.day41_inventory AS
SELECT site_id, count(*) AS
         equipment_count
FROM course.equipment
GROUP BY site_id;
```

This stores two inventory rows. `REFRESH MATERIALIZED VIEW student.day41_inventory;` reruns the defining query. This ordinary PostgreSQL feature is not a Timescale continuous aggregate.

SQL challenge

Materialize one bounded comfort or energy summary in `student`, preserving enough sums and weights to reconcile it. Keep a separate detail query for the corresponding scatter dataset. Write a short refresh procedure and an explicit statement of what the summary cannot reconstruct.

Experiment

Use a tiny scratch source table and materialized view. Insert a scratch source row, query the stored result, refresh, then query again. Predict both results. Do not mutate the course fixture merely to demonstrate staleness.

Acceptance checks

- The inventory example has counts two for north and one for south.
- Your scratch experiment visibly demonstrates stored-result staleness and refresh.
- Summary totals reconcile to the original detail query.
- The report records source window, policy and as-of evidence; a summary row is never offered as a scatter observation.

Stretch and reflection

Which source or policy changes require recomputation? Explain how you would detect that a stored result was produced with an obsolete quality policy.

Save SQL, actual output and a short interpretation in your own `student-work/sql/day41/` directory. Label evidence **observed**, **fixture-only**, or **not run**. Consult this week's primary references for exact language and domain details.

Previous: Day 40 ·

Day 42 — Week 6 independent challenge — an analytics engineering review

Previous: Day 41 · **Week 6 review · 2–4 hours, split across sessions as needed.** Prerequisites: Days 36–41, or equivalent skills.

Project brief

Prepare a small analytics package for a reviewer who knows SQL but has not seen your earlier lessons. Cover comfort, power versus OAT, and supply-air temperature error for all fixture AHUs. The package should make every metric's grain, units, eligibility and denominator discoverable.

Acceptance criteria

- Comfort includes all-time and fan-on denominators, coverage and the inclusive 20–24 °C teaching band.
- North Supply A's one-hour comfort excursion and north Supply B's one-hour

warm SAT interval remain attributable to their correct equipment and time.
- A scatter detail result has an explicit unique grain, source identity and valid duration weights.
- A summary preserves sums and denominators; its weighted means can be independently recomputed.
- A stored result has documented freshness and refresh behavior, demonstrated with scratch data.
- Any synthetic web-weather comparison retains source timestamps, bounded matching and a visible synthetic label.

Deliverables

Submit independently written SQL, a data dictionary, compact results and a brief review note. Include one unequal-weight counterexample and one join-multiplication counterexample. Explain why a visual association or out-of-range mixing fraction does not automatically justify a fault diagnosis.

Self-review

Could a reviewer reconstruct your percentage from exported columns? Could a refresh change a result without changing the source

window? Which distinctions disappear if only the final mean is retained? Identify one concrete improvement before presenting the package to another learner.

Label results **observed**, **fixture-only**, or **not run**. This page intentionally provides no worked solution or implementation sequence. Attempt the challenge independently before consulting the week's references.

Previous: Day 41 ·

Primary reading — Week 6

Use the lesson first, then these references for precise behavior. Full list: SOURCES.md.

Week 6

- Conditional expressions
- Statistical aggregates
- Materialized views
- Versioned analytics source